Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 4 – Laboratory Report

Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Name: Shravan P M
Roll No.: 24110026
Registration No.: 24011101101

1. Objective

The objectives of this experiment are

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance of different architectures.

2. Learning Outcomes

After completing this experiment, the following outcomes were achieved.

1. Explained modern CNN architectures and their design motivations.
2. Differentiated LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implemented transfer learning using a pretrained ImageNet backbone.
4. Fine tuned a pretrained CNN model by selectively unfreezing convolution layers.
5. Compared CNN architectures using performance metrics.

3. Introduction

Deep Convolutional Neural Networks have become the dominant models for computer vision applications because they automatically learn hierarchical features from images. The evolution of CNNs has focused on improving classification accuracy while reducing computational complexity and training difficulties.

The progression of CNN architectures began with LeNet-5 and has evolved through AlexNet, VGGNet, GoogleNet, and ResNet. Each architecture introduced important innovations that significantly improved image classification performance. This report implements and compares these architectures on the CIFAR-10 dataset using transfer learning.

4. Evolution of CNN Architectures

Model	Year	Major Contribution
LeNet-5	1998	First practical convolutional neural network
AlexNet	2012	ReLU activation, Dropout and GPU training
VGG16	2014	Uniform 3×3 convolution filters
GoogleNet	2014	Inception modules for multi-scale feature extraction
ResNet	2015	Residual learning using skip connections

5. LeNet-5

LeNet-5 was proposed by Yann LeCun in 1998 for handwritten digit recognition. It consists of two convolution layers, two average pooling layers and fully connected layers. It demonstrated that convolutional networks could automatically learn image features without handcrafted feature extraction.

Advantages

- Small number of parameters
- Fast training
- Suitable for OCR

6. AlexNet

AlexNet won the ImageNet Challenge in 2012 by reducing the classification error significantly. It introduced ReLU activation, Dropout regularization and GPU training.

Major innovations

- ReLU activation
- Dropout
- GPU implementation
- Data augmentation

7. VGG16

VGG16 demonstrated that using multiple small 3×3 convolution filters could outperform larger filters while increasing network depth.

Advantages

- Excellent feature extraction
- Simple architecture
- High classification accuracy

8. Comparison of Architectures

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters
GoogleNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning

9. GoogleNet (Inception Network)

GoogleNet, proposed by Szegedy *et al.* in 2014, introduced the **Inception Module**, which performs multiple convolution operations in parallel using different filter sizes. Instead of using only a single kernel size, GoogleNet simultaneously applies 1×1, 3×3, 5×5 convolutions and max pooling. The outputs are concatenated to obtain multi-scale feature representations.

The use of 1×1 convolutions before larger kernels reduces the number of trainable parameters and computational complexity.

Advantages

- Multi-scale feature extraction.
- Fewer parameters than VGG16.
- High computational efficiency.
- Better classification performance.

10. ResNet

Increasing CNN depth often causes the **vanishing gradient problem**, making optimization difficult. ResNet solves this issue using **skip (residual) connections**, allowing gradients to flow directly through the network.

Instead of learning

$$H(x)$$

ResNet learns

$$F(x) = H(x) - x$$

thus,

$$Output = F(x) + x$$

where
- $F(x)$ = Residual Mapping
- x = Identity Mapping

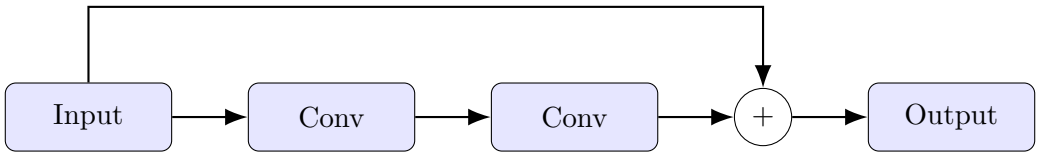


Figure 1: Residual Block in ResNet

Advantages
- Eliminates vanishing gradients.
- Enables very deep neural networks.
- Faster convergence.
- High ImageNet accuracy.

11. Dilated Convolution

Dilated convolution (also called atrous convolution) increases the receptive field without increasing the number of parameters.

The dilation rate determines the spacing between kernel elements.

For a standard convolution,

$$D = 1$$

For dilated convolution,

$$D > 1$$

Example

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 4 & 0 & 5 \end{bmatrix}$$

where zeros indicate inserted gaps.

Applications
- Semantic segmentation
- Medical image analysis
- Satellite imagery
- Object localization

12. Transpose Convolution

Transpose convolution increases the spatial dimensions of feature maps. Unlike pooling, which reduces image size, transpose convolution performs learnable upsampling.

Applications
- Image Super Resolution
- Autoencoders

- GANs
- Image Generation
- Semantic Segmentation

13. Transfer Learning

Transfer Learning utilizes a pretrained CNN model that has already learned general image features from a large dataset such as ImageNet.

Instead of training from scratch,

1. Load a pretrained model.
2. Remove the original classifier.
3. Freeze convolution layers.
4. Add new Dense layers.
5. Train the classifier.
6. Fine tune selected convolution layers.

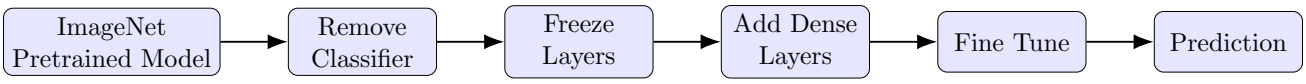


Figure 2: Transfer Learning Workflow

Advantages

- Faster convergence.
- Higher accuracy on small datasets.
- Reduced training time.
- Lower computational cost.

14. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Number of Classes : 10
- Image Size : $32 \times 32 \times 3$
- Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck.

15. Experimental Procedure

Task 1: Dataset Preparation

1. Loaded the CIFAR-10 dataset using `tf.keras.datasets.cifar10`.
2. Normalized the pixel values to the range [0,1].
3. Displayed ten sample images.
4. Printed the dimensions of the training and testing datasets.
   (50000×32×32×3 train, 10000×32×32×3 test)

Task 2: Transfer Learning

Architecture used: **VGG16** (pretrained on ImageNet).

Steps performed.

1. Loaded pretrained ImageNet weights.
2. Removed the original classification layer.
3. Froze the convolutional base.
4. Added a Global Average Pooling layer.
5. Added a Dense layer (256 units) with ReLU activation.
6. Added the output layer (10 units) with Softmax activation.

Task 3: Model Training

Trained the model using

Optimizer	Adam
Learning Rate	0.001
Batch Size	32
Epochs	15
Loss Function	Categorical Cross Entropy
Evaluation Metric	Accuracy

Task 4: Fine Tuning

1. Unfroze the last convolution block of VGG16.
2. Trained for another 8 epochs at a reduced learning rate (1×10^{-5}).
3. Compared the classification accuracy before and after fine tuning (see Section 18.1).

Task 5: Model Evaluation

Evaluated the trained model using

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

16. Hyperparameter Study

Compared performance using the following settings (see `run_hyperparameter_study()` in the accompanying script).

Hyperparameter	Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Epochs	10, 20
Optimizer	Adam, SGD
Dense Units	128, 256
Frozen Layers	All, Partial

17. Source Code

The complete source code for this experiment has been uploaded to a GitHub repository for reference. The repository is periodically updated with code refinements, additional implementations, and detailed insights gained through successive iterations of the experiment.

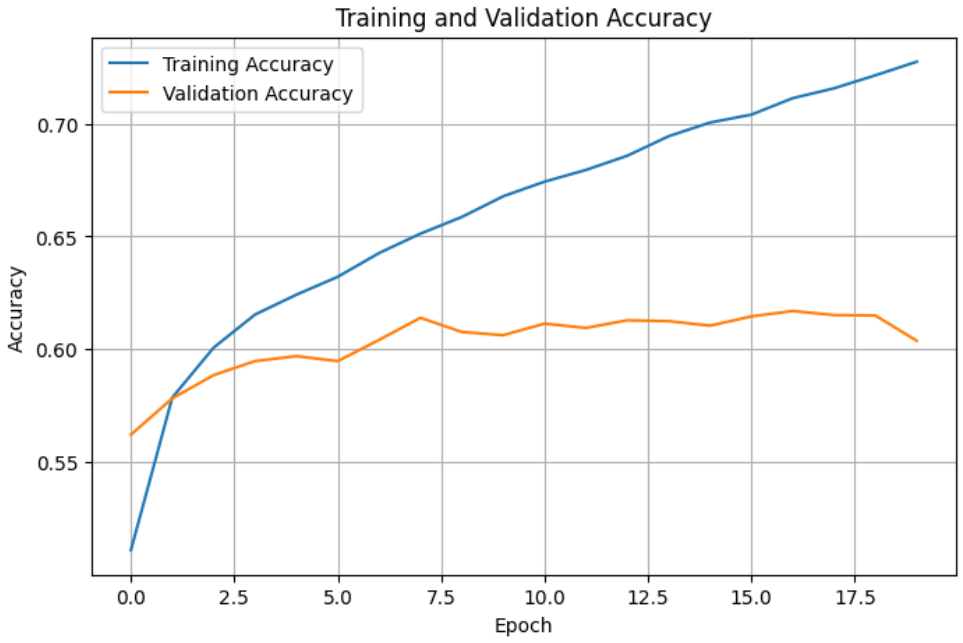
GitHub Repository:

`https://github.com/shravW/Deep-Learning-Lab-CS3807-SEM-5/tree/main/Ex_4_DL`

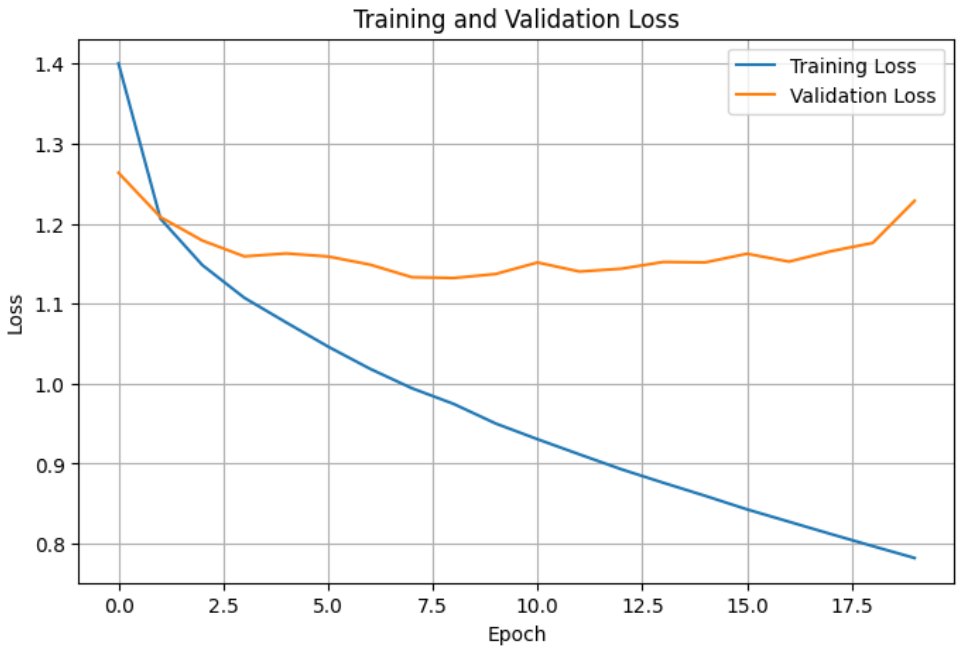
18. Mandatory Plots



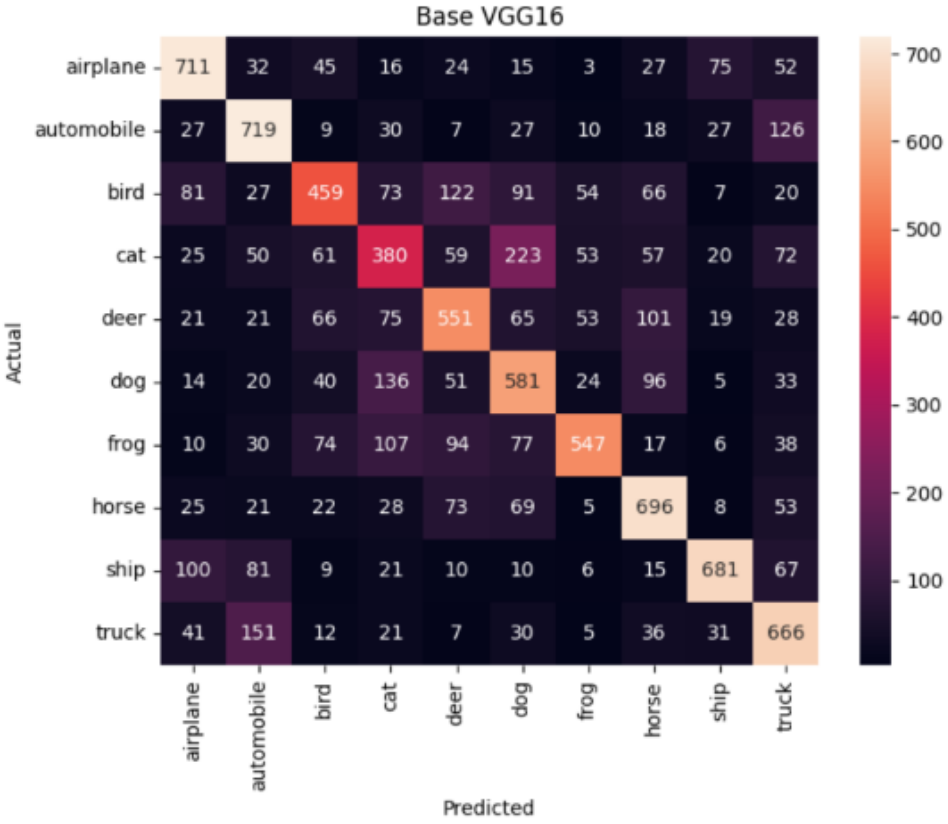
*T*he 32 × 32 resolution is too coarse to distinguish subtle differences between classes (e.g., cat vs. dog) even for a human observer.



*T*he model learns well at first, but later the training accuracy keeps improving while validation accuracy stops improving. This shows that the model is starting to overfit.



*T*raining loss steadily decreases, while validation loss starts increasing after around epoch 8. This indicates that the model begins to overfit the training data after this point.



*T*he model performs well on some classes but struggles to distinguish between similar classes. Most confusion occurs between cat and dog, while classes like automobile, ship, and horse are classified more accurately.

19. Results

19.1 Performance Metrics

Metric	Value
Training Accuracy (final epoch)	0.7275
Testing Accuracy (before fine tuning)	0.5996
Testing Accuracy (after fine tuning)	0.7311
Precision (weighted)	0.6010
Recall (weighted)	0.5997
F1-score (weighted)	0.5923
Training Time - Task 3 (frozen)	117.1 s
Training Time - Task 4 (fine tuning)	145.3 s
Training Time - Total	262.4 s
Total Parameters	14,781,642
Trainable Parameters	66,954

19.2 Comparison of CNN Architectures

Model	Parameters	Accuracy (%)	Training Time
LeNet-5 (scratch)	62006	57.85	71.8
AlexNet (scratch)	2012298	74.08	132.6
VGG16 (transfer + fine-tune)	14781642	73.11	262.4
ResNet50 (transfer + fine-tune)	23851274	46.66	251.4

20. Discussion Questions

1. **Why is AlexNet considered a breakthrough in deep learning?**
   AlexNet was the first deep CNN to achieve a major breakthrough in the 2012 ImageNet challenge, showing that deep learning could outperform traditional computer-vision methods. It also introduced techniques like ReLU activations, Dropout, and GPU-based training, which later became widely used in deep learning architectures.
2. **Why does VGG16 use only 3×3 convolution filters?**
   Using several smaller 3×3 convolutions can achieve the same receptive field as larger filters like 5×5 or 7×7. This uses fewer parameters and adds more activation layers, making the network more efficient and improving its ability to learn complex features.
3. **Explain the advantages of the Inception module.**
   The Inception module uses different filter sizes and pooling at the same time to detect features at different scales. 1×1 convolutions reduce the number of channels before larger filters, making the network more efficient while maintaining good performance.
4. **What is the purpose of residual learning?**
   Residual learning allows a block to learn the difference between the desired output and the input, then adds the input back using a skip connection. This makes training easier, improves gradient flow, and allows very deep networks to be trained effectively.
5. **Differentiate LeNet and ResNet.**
   LeNet-5 is a simple 7-layer network with around 60K parameters, designed for small grayscale images. ResNet50 is much deeper, with around 25.6M parameters, and uses ReLU, batch normalization, and residual connections to make training deep networks easier.
6. **What is Transfer Learning?**
   Transfer learning means using a model that has already been trained on a large dataset as a starting point for a new task. This saves training time and allows the model to use features it has already learned.

7. **Why is fine tuning required?**
   A pretrained model can provide useful features, but they may not be perfectly suited to the new dataset. Fine-tuning means unfreezing some later layers and training them with a small learning rate so the model can adapt to the new task and improve accuracy.
8. **Explain the difference between dilated convolution and transpose convolution.**
   Dilated convolution increases the receptive field by adding gaps between kernel elements without increasing the number of parameters. Transpose convolution is used for learnable upsampling, increasing the image size for tasks such as segmentation and image generation.
9. **Why do pretrained models converge faster?**
   Pretrained models already have useful features like edges, textures, and shapes learned from large datasets. This means only the classifier and a few later layers need training, making the process faster and requiring fewer updates than training from scratch.
10. **Compare the computational complexity of LeNet and ResNet.** LeNet-5 is a small network with around 60K parameters and 7 layers, so it is quick to train and run. ResNet50 is much larger, with around 25.6M parameters and 50 layers, requiring more computational power and taking longer for training and inference.

21. Conclusion

This experiment studied the development of CNN architectures from LeNet-5 to ResNet and explored transfer learning using a pretrained VGG16 model for CIFAR-10 classification. The pretrained VGG16 model achieved good performance while requiring less training compared to building a model from scratch, and fine-tuning the convolutional layers further improved its accuracy. The comparison of different architectures showed how deeper and more advanced networks can learn more complex image features. Overall, the experiment demonstrated the importance of network architecture, transfer learning, and fine-tuning in improving CNN performance for image classification tasks.

22. References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, *ImageNet Classification with Deep Convolutional Neural Networks*, NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR, 2015.
4. Christian Szegedy et al., *Going Deeper with Convolutions*, CVPR, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, *Deep Residual Learning for Image Recognition*, CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
7. TensorFlow Documentation: `https://www.tensorflow.org`
8. Keras Documentation: `https://keras.io`